configurationProxy HTTP/TLS/JSON

Interface Design Description

Service ID: *"configurationProxy"*

Abstract
This document describes the HTTP/1.1 protocol with TLS payload security and JSON payload encoding variant of the `configurationProxy` service of the Configuration system.

Contents

1 Documentation edition and implementation scope **3**

2 Overview **3**
2.1 Interface registration . . . 3

3 Service Operations **4**
3.1 operation get-configuration . . . 4
3.2 operation store-configuration . . . 4

4 Data Models **6**
4.1 struct ConfigurationRequest . . . 6
4.2 struct ConfigurationResponse . . . 6
4.3 struct ErrorMessage . . . 6
4.4 Primitives . . . 6

5 Go and Rust implementation variants **8**
5.1 Go implementation . . . 8
5.2 Rust implementation . . . 8

6 References **8**

7 Revision History **10**
7.1 Amendments . . . 10
7.2 Quality Assurance . . . 10

1 Documentation edition and implementation scope

This is the 5.3.0 documentation edition, dated 2026-11-09. It carries forward the system and service contracts from the retained 5.2.2 documentation and identifies the current primary implementation sources. The former `documentation/v5.2.2` folder is retained unchanged.

Document version 5.3.0 is distinct from executable, dependency, registered system/service and wire-contract versions. Values still defined by the source (including 5.2.2 artefact versions, 1.0.0 service versions and `SysMonitor-AMQP-IDD-5.2.2` payload identifiers) are preserved. This documentation update does not change runtime behaviour or establish a new implementation release. Historical verification statements describe the tests recorded in the inherited baseline; this revision validates document compilation and references.

2 Overview

This document describes the `configurationProxy` service interface of the Configuration system, which provides reading and storing configurations on behalf of other systems.

It's implemented using protocol, encoding as stated in below table:

Profile type	Type	Version
Transfer protocol	HTTP	1.1 [1]
Data encryption	TLS	1.3 [2], 1.2 [3]
Encoding	JSON	RFC 8259 [4]
Compression	N/A	-
Semantics	N/A	-
Ontology	N/A	-

Table 1: Communication and semantics details used for the `configurationProxy` service interface

This document provides the Interface Design Description IDD to the *configurationProxy – Service Description* document [5]. For further details about how this service is meant to be used, please consult that document.

The rest of this document describes how to realize the `configurationProxy` service interface in detail. Both in terms of its operations (Section 3) and its information model (Section 4).

2.1 Interface registration

The service is registered in the ServiceRegistry with the interface template `generic_https`, policy `CERT_AUTH`, access address `configuration`, port 8459 (`server.port`; the standalone deployment registers 8450) and base path `/configuration/proxy`. The `operations` property of the interface lists the operations below with their method and path relative to the base path (path parameters are placeholders such as `{id}` that the consumer fills in):

Operation	Method	Path
`get-configuration`	GET	`/config/{systemName}`
`store-configuration`	PUT	`/save-config`

3 Service Operations

The interfaces of the `configurationProxy` service, its operations, data models and implementation are provided below. In particular, each subsection first names the HTTP method and path used to call the operation, after which it names the abstract operation from the `configurationProxy` SD document, as well as input and output types.

All operations require the requester's identity, presented according to the provider's authentication policy: the client certificate (mutual TLS) and, with the `outsourced` policy, `Authorization: Bearer IDENTITY-TOKEN//<token>`; systems holding the Authentication secret key send `Bearer AUTHENTICATOR-KEY//<system>//<hash>`, and with the `declared` policy `Bearer SYSTEM//<system>` is sent. Request and response bodies are `application/json` unless stated otherwise. Errors are answered with an ErrorMessage body and the status codes `400 Bad Request` (invalid parameter), `401 Unauthorized` (missing or invalid identity), `403 Forbidden` (not permitted, e.g. by the management, authorization or blacklist filter), `404 Not Found`, `408 Request Timeout`, `423 Locked`, `500 Internal Server Error` and `503 Service Unavailable` (a system it depends on failed).

3.1 GET /configuration/proxy/config/*{systemName}*

Operation: get-configuration
Input: String
Output: ConfigurationResponse

Returns the configuration of the named system.

Path parameter: *systemName*.

A successful call returns `200 OK` with the output in the body.

```
GET /configuration/proxy/config/<systemName> HTTP/1.1
Host: configuration:8459
Authorization: Bearer IDENTITY-TOKEN//<token>
```

Listing 1: An get-configuration invocation.

```
{
  "id": 42,
  "systemName": "TemperatureSensor",
  "fileName": "thermostat.json",
  "contentType": "application/json",
  "data": "eyJzZXRwb2ludCI6MjF9",
  "size": 10,
  "createdAt": "2026-10-22T12:00:00Z",
  "updatedAt": "2026-10-22T12:00:00Z"
}
```

Listing 2: An get-configuration response.

3.2 PUT /configuration/proxy/save-config

Operation: store-configuration
Input: ConfigurationRequest
Output: ConfigurationResponse

Stores the configuration of the system named in the body.

A successful call returns `200 OK` with the output in the body.

```
PUT /configuration/proxy/save-config HTTP/1.1
Host: configuration:8459
Authorization: Bearer IDENTITY-TOKEN//<token>
```

```
Content-Type: application/json

{
  "systemName": "TemperatureSensor",
  "fileName": "thermostat.json",
  "contentType": "application/json",
  "data": "eyJzZXRwb2ludCI6MjF9"
}
```

Listing 3: An store-configuration invocation.

```
{
  "id": 42,
  "systemName": "TemperatureSensor",
  "fileName": "thermostat.json",
  "contentType": "application/json",
  "data": "eyJzZXRwb2ludCI6MjF9",
  "size": 10,
  "createdAt": "2026-10-22T12:00:00Z",
  "updatedAt": "2026-10-22T12:00:00Z"
}
```

Listing 4: An store-configuration response.

4 Data Models

Here, all data objects that can be part of the service calls associated with this service are listed in alphabetic order. Note that each subsection, which describes one type of object, begins with the *struct* keyword, which is meant to denote a JSON Object that must contain certain fields, or names, with values conforming to explicitly named types. Fields that are not needed may be omitted or `null` in requests; empty fields are omitted in responses. As a complement to the primary types defined in this section, there is also a list of secondary types in Section 4.4, which are used to represent things like hashes, identifiers and texts.

4.1 struct ConfigurationRequest

A configuration file to store.

Object Field	Value Type	Description
"systemName"	String	System the configuration belongs to (proxy: required; management: taken from the path).
"fileName"	String	Plain file name of the configuration.
"contentType"	String	Media type of the configuration file.
"data"	String	File content, Base64 (at most `configuration.max.size` bytes decoded).

4.2 struct ConfigurationResponse

A stored configuration file.

Object Field	Value Type	Description
"id"	Number	Identifier of the stored configuration.
"systemName"	String	System the configuration belongs to.
"fileName"	String	Plain file name of the configuration.
"contentType"	String	Media type of the configuration file.
"data"	String	File content, Base64.
"size"	Number	Size of the decoded file in bytes.
"createdAt"	DateTime	UTC time of creation.
"updatedAt"	DateTime	UTC time of the last change.

4.3 struct ErrorMessage

Object Field	Value Type	Description
"errorMessage"	String	Error text, if the event is an error.
"errorCode"	Number	
"exceptionType"	ExceptionType	
"origin"	String	

4.4 Primitives

As all messages are encoded using the JSON format [4], the following primitive constructs, part of that standard, become available. Note that the official standard is defined in terms of parsing rules, while this list only concerns syntactic information. Furthermore, the Object and Array types are given optional generic type parameters,

which are used in this document to signify when pair values or elements are expected to conform to certain types.

Type	Description
Array	A JSON Array [4]; Array<A> has elements of type A.
DateTime	JSON String with an ISO 8601 UTC timestamp, e.g. `2026-10-22T12:00:00Z`.
ExceptionType	One of `ARROWHEAD`, `INVALID_PARAMETER`, `AUTH`, `FORBIDDEN`, `DATA_NOT_FOUND`, `TIMEOUT`, `LOCKED`, `INTERNAL_SERVER_ERROR`, `EXTERNAL_SERVER_ERROR` (JSON String).
Number	JSON Number, an integer.
Object	JSON Object; Object<A> has string keys and values of type A.
String	JSON String.

5 Go and Rust implementation variants

5.1 Go implementation

Primary system folder: `Eclipse-Arrowhead-go/ah5-Configuration-go`.

The system owns its Go module, launcher and build configuration. Use the per-system `Makefile` and `go.mod`; the containing Go tree is the build source, not a historical Arrowhead copy. Ordinary native roles use independent identities and JSON snapshot state rather than the Java database. Do not assume Java SQL migration, clustering or identical transaction/retry semantics. The registry has its own configuration and optional native AMQP monitor; other roles must be checked against their own registrations.

Listening port. Built-in default 127.0.0.1:8459 (Arrowhead default). The shipped config.json, compose.yaml and Dockerfile use the same port on all interfaces ("listen": "0.0.0.0:8459"; 8540–8555 layout before 2026-11-09). Override with "listen" in the configuration file, SERVER_PORT=N or -server.port N; the port overrides keep the configured host.

Native behavior and outstanding work. Local configuration CRUD/base64/raw download and local proxy save/get. Java external-provider fallback and full DTO/error parity. These are the boundaries recorded by the native compatibility document; later implementation changes still require direct qualification. Native design/route evidence: `Eclipse-Arrowhead-go/ah5-Configuration-go/documentation/Configuration`.

For this Java interface, select only a native transport and operation explicitly implemented and advertised by that port. A matching method/path does not establish DTO, error, authorization, retry or persistence equivalence. Java MQTT/AMQP bindings are not automatically provided by a Go or Rust role. Differential contract and security checks remain necessary before substitution.

5.2 Rust implementation

Primary system folder: `Eclipse-Arrowhead-rust/ah5-Configuration-rust`.

The per-system entry point is declared as a binary of `arrowhead-support-rust` in `ah5-support-runtime-rust/Cargo.toml`. Build from the primary Rust workspace with the applicable package/binary selection. Documentation links can resolve into the shared core/support catalogue. Independent process identity and state remain required.

The ordinary native profile uses Tokio/Axum and rustls with peer-identity checks and independent locked JSON state. Network effects are not distributed database transactions. The precise trust, CRL reload and persistence guarantees are those of the native profile, not the Java implementation.

Listening port. Default 127.0.0.1:8459 (Arrowhead default, role configuration). Override with "listen": "host:port" in the configuration, SERVER_PORT=N or –server.port N; the port overrides keep the configured host.

Native behavior and outstanding work. Durable base64/raw storage, owner-constrained proxy APIs. External fallback/discovery, complete MIME validation. These are the boundaries recorded by the native compatibility document; later implementation changes still require direct qualification. Native design/route evidence: `Eclipse-Arrowhead-rust/ah5-support-runtime-rust/documentation/Configuration`.

For this Java interface, select only a native transport and operation explicitly implemented and advertised by that port. A matching method/path does not establish DTO, error, authorization, retry or persistence equivalence. Java MQTT/AMQP bindings are not automatically provided by a Go or Rust role. Differential contract and security checks remain necessary before substitution.

6 References

[1] R. Fielding, M. Nottingham, and J. Reschke, “Http/1.1,” IETF, RFC 9112, 2022.

[2] E. Rescorla, “The transport layer security (tls) protocol version 1.3,” IETF, RFC 8446, 2018.

[3] T. Dierks and E. Rescorla, “The transport layer security (tls) protocol version 1.2,” IETF, RFC 5246, 2008.

[4] T. Bray, “The javascript object notation (json) data interchange format,” IETF, RFC 8259, 2017.

[5] J. Delsing, “configurationProxy – Service Description (Configuration),” Eclipse Arrowhead, SD 5.3.0, 2026, configuration-v5.3.0-configurationProxy-SD.pdf.

7 Revision History

7.1 Amendments

No.	Date	Version	Subject of Amendments	Author
1	2026-10-22	5.2.2	Document created from the Arrowhead LaTeX templates for the 5.2.2 Java Spring implementation, including the local extensions of this workspace.	Jerker Delsing
2	2026-10-30	5.2.2	Checked against the primary Java source tree (Eclipse-Arrowhead-java): standalone deployment, configuration and security statements updated; missing primitive definitions added.	Claude Opus 5.5 (draft)
3	2026-11-09	5.3.0	New documentation edition; primary-source references and Go/Rust implementation scope; 5.2.2 retained.	Documentation maintenance

7.2 Quality Assurance

No.	Date	Version	Approved by
1		5.3.0	